

Disaster Management System

DBMS Mini Project-SQL , PL/SQL Queries

UCE2024602 – Aditi Singnurkar

UCE2024607 – Ankita Nahire

UCE2024613 – Jui Baviskar

UCE2024614 – Bhoomi Shekapure

CREATE TABLE QUERIES

1. Disaster

```
CREATE TABLE Disaster (  
    Disaster_ID INT PRIMARY KEY,  
    Disaster_Type VARCHAR(100) NOT NULL,  
    Location VARCHAR(100),  
    Start_Date DATE,  
    End_Date DATE,  
    Severity_Level VARCHAR(50)  
);
```

2. Shelter

```
CREATE TABLE Shelter (  
    Shelter_ID INT PRIMARY KEY,  
    Address VARCHAR(150),  
    Capacity INT,  
    Coordinator_Name VARCHAR(100),  
    Disaster_ID INT,  
    FOREIGN KEY (Disaster_ID)  
        REFERENCES Disaster(Disaster_ID)  
        ON DELETE CASCADE  
);
```

3. Resource

```
CREATE TABLE Resource (  
    Resource_ID INT PRIMARY KEY,  
    Resource_Name VARCHAR(100) NOT NULL,  
    Description VARCHAR(200)
```

);

4. Volunteer

```
CREATE TABLE Volunteer (  
    Volunteer_ID INT PRIMARY KEY,  
    Name VARCHAR(100) NOT NULL,  
    Availability VARCHAR(50),  
    Role VARCHAR(50),  
    Shift_Timing VARCHAR(50)  
);
```

5. Victim

```
CREATE TABLE Victim (  
    Victim_ID INT PRIMARY KEY,  
    Name VARCHAR(100),  
    Age INT,  
    Contact VARCHAR(15),  
    Disaster_ID INT,  
    Shelter_ID INT,  
    Medical_Needs VARCHAR(200),  
    FOREIGN KEY (Disaster_ID)  
        REFERENCES Disaster(Disaster_ID)  
        ON DELETE CASCADE,  
    FOREIGN KEY (Shelter_ID)  
        REFERENCES Shelter(Shelter_ID)  
        ON DELETE CASCADE  
);
```

6. Volunteer_Assignments

```
CREATE TABLE Volunteer_Assignments (  
    Volunteer_ID INT,  
    Disaster_ID INT,  
    Role VARCHAR(50),  
    Shift_Timing VARCHAR(50),  
    PRIMARY KEY (Volunteer_ID, Disaster_ID),  
    FOREIGN KEY (Volunteer_ID)  
        REFERENCES Volunteer(Volunteer_ID)  
        ON DELETE CASCADE,  
    FOREIGN KEY (Disaster_ID)  
        REFERENCES Disaster(Disaster_ID)  
        ON DELETE CASCADE  
);
```

7. Volunteer_Skills

```
CREATE TABLE Volunteer_Skills (  

```

```

    Volunteer_ID INT,
    Skill VARCHAR(100),
    PRIMARY KEY (Volunteer_ID, Skill),
    FOREIGN KEY (Volunteer_ID)
        REFERENCES Volunteer(Volunteer_ID)
        ON DELETE CASCADE
);

```

8. Incident_Report

```

CREATE TABLE Incident_Report (
    Report_ID INT PRIMARY KEY,
    Description VARCHAR(255),
    Timestamp DATETIME,
    Volunteer_ID INT,
    Disaster_ID INT,
    FOREIGN KEY (Volunteer_ID)
        REFERENCES Volunteer(Volunteer_ID)
        ON DELETE CASCADE,
    FOREIGN KEY (Disaster_ID)
        REFERENCES Disaster(Disaster_ID)
        ON DELETE CASCADE
);

```

9. Shelter_Inventory

```

CREATE TABLE Shelter_Inventory (
    Shelter_ID INT,
    Resource_ID INT,
    Quantity INT,
    PRIMARY KEY (Shelter_ID, Resource_ID),
    FOREIGN KEY (Shelter_ID)
        REFERENCES Shelter(Shelter_ID)
        ON DELETE CASCADE,
    FOREIGN KEY (Resource_ID)
        REFERENCES Resource(Resource_ID)
        ON DELETE CASCADE
);

```

10. Resource_Location

```

CREATE TABLE Resource_Location (
    Resource_ID INT,
    Disaster_ID INT,
    Shelter_ID INT,
    Quantity INT,
    PRIMARY KEY (Resource_ID, Disaster_ID, Shelter_ID),
    FOREIGN KEY (Resource_ID)

```

```

        REFERENCES Resource(Resource_ID)
        ON DELETE CASCADE,
FOREIGN KEY (Disaster_ID)
        REFERENCES Disaster(Disaster_ID)
        ON DELETE CASCADE,
FOREIGN KEY (Shelter_ID)
        REFERENCES Shelter(Shelter_ID)
        ON DELETE CASCADE
);

```

11. Incident_Volunteer_Details

```

CREATE TABLE Incident_Volunteer_Details (
    Report_ID INT PRIMARY KEY,
    Vol_Name VARCHAR(100),
    FOREIGN KEY (Report_ID)
        REFERENCES Incident_Report(Report_ID)
        ON DELETE CASCADE
);

```

12. Incident_Disaster_Details

```

CREATE TABLE Incident_Disaster_Details (
    Report_ID INT PRIMARY KEY,
    Disaster_Type VARCHAR(100),
    FOREIGN KEY (Report_ID)
        REFERENCES Incident_Report(Report_ID)
        ON DELETE CASCADE
);

```

13. Victim_Shelter_Details

```

CREATE TABLE Victim_Shelter_Details (
    Victim_ID INT PRIMARY KEY,
    Shelter_Address VARCHAR(150),
    FOREIGN KEY (Victim_ID)
        REFERENCES Victim(Victim_ID)
        ON DELETE CASCADE
);

```

14. Victim_Disaster_Details

```

CREATE TABLE Victim_Disaster_Details (
    Victim_ID INT PRIMARY KEY,
    Disaster_Type VARCHAR(100),
    FOREIGN KEY (Victim_ID)
        REFERENCES Victim(Victim_ID)
        ON DELETE CASCADE
);

```

15. Volunteer_Assignment_Names

```
CREATE TABLE Volunteer_Assignment_Names (  
    Volunteer_ID INT PRIMARY KEY,  
    Name VARCHAR(100),  
    FOREIGN KEY (Volunteer_ID)  
        REFERENCES Volunteer(Volunteer_ID)  
        ON DELETE CASCADE  
);
```

16. Volunteer_Assignment_Disaster_Types

```
CREATE TABLE Volunteer_Assignment_Disaster_Types (  
    Disaster_ID INT PRIMARY KEY,  
    Disaster_Type VARCHAR(100),  
    FOREIGN KEY (Disaster_ID)  
        REFERENCES Disaster(Disaster_ID)  
        ON DELETE CASCADE  
);
```

17. Shelter_Inventory_Shelter_Details

```
CREATE TABLE Shelter_Inventory_Shelter_Details (  
    Shelter_ID INT PRIMARY KEY,  
    Shelter_Address VARCHAR(150),  
    Coord_Name VARCHAR(100),  
    Disaster_ID INT,  
    FOREIGN KEY (Shelter_ID)  
        REFERENCES Shelter(Shelter_ID)  
        ON DELETE CASCADE,  
    FOREIGN KEY (Disaster_ID)  
        REFERENCES Disaster(Disaster_ID)  
        ON DELETE CASCADE  
);
```

18. Shelter_Inventory_Resource_Details

```
CREATE TABLE Shelter_Inventory_Resource_Details (  
    Resource_ID INT PRIMARY KEY,  
    Resource_Name VARCHAR(100),  
    FOREIGN KEY (Resource_ID)  
        REFERENCES Resource(Resource_ID)  
        ON DELETE CASCADE  
);
```

19. Shelter_Inventory_Disaster_Details

```
CREATE TABLE Shelter_Inventory_Disaster_Details (  
    Shelter_ID INT,
```

```

Disaster_ID INT,
Disaster_Type VARCHAR(100),
PRIMARY KEY (Shelter_ID, Disaster_ID),
FOREIGN KEY (Shelter_ID)
    REFERENCES Shelter(Shelter_ID)
    ON DELETE CASCADE,
FOREIGN KEY (Disaster_ID)
    REFERENCES Disaster(Disaster_ID)
    ON DELETE CASCADE
);

```

QUERIES:

1. List all disasters and their total number of shelters.

```

mysql> SELECT d.Disaster_ID, d.Disaster_Type, COUNT(s.Shelter_ID) AS Total_Shelters
-> FROM Disaster d
-> LEFT JOIN Shelter s ON d.Disaster_ID = s.Disaster_ID
-> GROUP BY d.Disaster_ID, d.Disaster_Type;

```

Disaster_ID	Disaster_Type	Total_Shelters
1	Earthquake	1
2	Flood	1
3	Cyclone	1
4	Landslide	1
5	Fire	1
6	Drought	1
7	Tsunami	1
8	Chemical Leak	1
9	Building Collapse	1
10	Forest Fire	0

10 rows in set (0.10 sec)

2. Show each victim's name, contact, shelter address, and disaster type.

```

mysql> SELECT v.Name, v.Contact, s.Address, d.Disaster_Type
-> FROM Victim v
-> JOIN Shelter s ON v.Shelter_ID = s.Shelter_ID
-> JOIN Disaster d ON v.Disaster_ID = d.Disaster_ID;

```

Name	Contact	Address	Disaster_Type
Ravi Kumar	9876543210	123 Relief Road, Pune	Earthquake
Sunita Devi	9823456789	Sector 9 Camp, Mumbai	Flood
Arjun Singh	9865321470	Green Valley Hall, Chennai	Cyclone
Lata Mishra	9854789652	Peace Center, Darjeeling	Landslide
Imran Ali	9990011223	City Hall, Delhi	Fire
Meena Patil	9911223344	Sunrise School, Nagpur	Drought
Akhil Verma	9001122334	Ocean View Shelter, Vizag	Tsunami
Kavita Rao	9023456712	Community Center, Vadodara	Chemical Leak
Shivam Das	9910099887	Kolkata Central Camp	Building Collapse

9 rows in set (0.01 sec)

3. Display volunteers assigned to a given disaster (ID = 1).

```
mysql> SELECT va.Volunteer_ID, vol.Name, va.Role, va.Shift_Timing
-> FROM Volunteer_Assignments va
-> JOIN Volunteer vol ON va.Volunteer_ID = vol.Volunteer_ID
-> WHERE va.Disaster_ID = 1;
```

Volunteer_ID	Name	Role	Shift_Timing
1	Aditi Kulkarni	Medical Support	Morning
4	Vikram Deshmukh	Rescue	Night

2 rows in set (0.00 sec)

4. List resources available in each shelter with quantities.

```
mysql> SELECT s.Shelter_ID, s.Address, r.Resource_Name, si.Quantity
-> FROM Shelter_Inventory si
-> JOIN Shelter s ON si.Shelter_ID = s.Shelter_ID
-> JOIN Resource r ON si.Resource_ID = r.Resource_ID
-> ORDER BY s.Shelter_ID;
```

Shelter_ID	Address	Resource_Name	Quantity
1	123 Relief Road, Pune	Food Packets	500
1	123 Relief Road, Pune	Water Bottles	300
2	Sector 9 Camp, Mumbai	Blankets	200
3	Green Valley Hall, Chennai	First Aid Kits	150
4	Peace Center, Darjeeling	Power Generators	20
5	City Hall, Delhi	Tents	100
6	Sunrise School, Nagpur	Clothes	80
7	Ocean View Shelter, Vizag	Sanitation Kits	120
8	Community Center, Vadodara	Flashlights	75
9	Kolkata Central Camp	Fuel Cans	50

10 rows in set (0.02 sec)

5. Show total quantity of every resource across all shelters.

```
mysql> SELECT r.Resource_Name, SUM(si.Quantity) AS Total_Quantity
-> FROM Resource r
-> JOIN Shelter_Inventory si ON r.Resource_ID = si.Resource_ID
-> GROUP BY r.Resource_Name;
```

Resource_Name	Total_Quantity
Food Packets	500
Water Bottles	300
Blankets	200
First Aid Kits	150
Power Generators	20
Tents	100
Clothes	80
Sanitation Kits	120
Flashlights	75
Fuel Cans	50

10 rows in set (0.00 sec)

6. Count disasters handled by each volunteer.

```
mysql> SELECT v.Name, COUNT(va.Disaster_ID) AS Disaster_Count
-> FROM Volunteer v
-> JOIN Volunteer_Assignments va ON v.Volunteer_ID = va.Volunteer_ID
-> GROUP BY v.Name;
+-----+-----+
| Name          | Disaster_Count |
+-----+-----+
| Aditi Kulkarni | 1              |
| Vikram Deshmukh | 1              |
| Rajesh Patel   | 1              |
| Neha Joshi     | 1              |
| Sara Khan      | 1              |
| Manoj Sharma   | 1              |
| Tina Mehta     | 1              |
| Ravi Bhosale   | 1              |
| Nikita Jain    | 1              |
| Suresh Reddy   | 1              |
+-----+-----+
10 rows in set (0.00 sec)
```

7. Get all victims affected by specific disaster type.

```
mysql> SELECT v.Name, v.Age, v.Contact, vdd.Disaster_Type
-> FROM Victim v
-> JOIN Victim_Disaster_Details vdd ON v.Victim_ID = vdd.Victim_ID
-> WHERE vdd.Disaster_Type = 'Earthquake';
+-----+-----+-----+-----+
| Name      | Age | Contact | Disaster_Type |
+-----+-----+-----+-----+
| Ravi Kumar | 35  | 9876543210 | Earthquake    |
+-----+-----+-----+-----+
1 row in set (0.01 sec)
```

8. Retrieve incidents for a specific disaster.

```
mysql> SELECT ir.Report_ID, ir.Description, ir.Timestamp, v.Name AS Reported_By
-> FROM Incident_Report ir
-> JOIN Volunteer v ON ir.Volunteer_ID = v.Volunteer_ID
-> WHERE ir.Disaster_ID = 2
-> ORDER BY ir.Timestamp DESC;
+-----+-----+-----+-----+
| Report_ID | Description          | Timestamp          | Reported_By |
+-----+-----+-----+-----+
| 2         | Flooded area needs more boats | 2025-08-12 09:00:00 | Rajesh Patel |
+-----+-----+-----+-----+
1 row in set (0.01 sec)
```

9. List volunteers not assigned to any disaster.




```
mysql> SELECT * FROM Volunteer
-> WHERE Volunteer_ID NOT IN (
->     SELECT DISTINCT Volunteer_ID FROM Volunteer_Assignments
-> );
```

Volunteer_ID	Name	Availability
12	Rohini Kulrkani	Available

1 row in set (0.01 sec)

10. Average victim age per disaster.

```
mysql> SELECT d.Disaster_Type, ROUND(AVG(v.Age), 2) AS Avg_Age
-> FROM Victim v
-> JOIN Disaster d ON v.Disaster_ID = d.Disaster_ID
-> GROUP BY d.Disaster_Type;
```

Disaster_Type	Avg_Age
Earthquake	35.00
Flood	42.00
Cyclone	28.00
Landslide	50.00
Fire	22.00
Drought	29.00
Tsunami	31.00
Chemical Leak	40.00
Building Collapse	26.00

9 rows in set (0.01 sec)

11. Top 3 shelters by capacity.

```
mysql> SELECT Shelter_ID, Address, Capacity
-> FROM Shelter
-> ORDER BY Capacity DESC
-> LIMIT 3;
```

Shelter_ID	Address	Capacity
7	Ocean View Shelter, Vizag	400
2	Sector 9 Camp, Mumbai	300
9	Kolkata Central Camp	270

3 rows in set (0.00 sec)

12. Victims whose shelters contain "Food Packets".

```
mysql> SELECT v.Name, s.Address, r.Resource_Name
-> FROM Victim v
-> JOIN Shelter s ON v.Shelter_ID = s.Shelter_ID
-> JOIN Shelter_Inventory si ON s.Shelter_ID = si.Shelter_ID
-> JOIN Resource r ON si.Resource_ID = r.Resource_ID
-> WHERE r.Resource_Name = 'Food Packets';
```

Name	Address	Resource_Name
Ravi Kumar	123 Relief Road, Pune	Food Packets

1 row in set (0.01 sec)

13. Calculate average age of victims.

```
mysql> SELECT AVG(Age) AS Average_Age FROM Victim;
-> //
```

Average_Age
33.6667

1 row in set (0.00 sec)

14. Total resources supplied to each disaster.

```
mysql> SELECT d.Disaster_Type, r.Resource_Name, SUM(rl.Quantity) AS Total_Supplied
-> FROM Resource_Location rl
-> JOIN Disaster d ON rl.Disaster_ID = d.Disaster_ID
-> JOIN Resource r ON rl.Resource_ID = r.Resource_ID
-> GROUP BY d.Disaster_Type, r.Resource_Name;
```

Disaster_Type	Resource_Name	Total_Supplied
Earthquake	Food Packets	500
Flood	Water Bottles	800
Cyclone	Blankets	250
Landslide	First Aid Kits	100
Fire	Power Generators	10
Drought	Tents	60
Tsunami	Clothes	90
Chemical Leak	Sanitation Kits	40
Building Collapse	Flashlights	30

9 rows in set (0.01 sec)

15. All incidents with disaster and volunteer names.

```
mysql> SELECT ir.Report_ID, ir.Description, d.Disaster_Type, v.Name AS Volunteer
-> FROM Incident_Report ir
-> JOIN Disaster d ON ir.Disaster_ID = d.Disaster_ID
-> JOIN Volunteer v ON ir.Volunteer_ID = v.Volunteer_ID;
```

Report_ID	Description	Disaster_Type	Volunteer
1	Minor aftershock reported	Earthquake	Aditi Kulkarni
2	Flooded area needs more boats	Flood	Rajesh Patel
3	Cyclone shelter power failure	Cyclone	Neha Joshi
4	Landslide blocked entry road	Landslide	Vikram Deshmukh
5	Fire under control	Fire	Sara Khan
6	Water shortage in Nagpur shelter	Drought	Manoj Sharma
7	Communication failure at coast	Tsunami	Tina Mehta
8	Gas leak spreading fast	Chemical Leak	Ravi Bhosale
9	Collapsed building fully evacuated	Building Collapse	Nikita Jain
10	Forest fire spreading eastward	Forest Fire	Suresh Reddy

10 rows in set (0.01 sec)

16. Volunteers not assigned to any disaster.

```
mysql> SELECT v.Volunteer_ID, v.Name
-> FROM Volunteer v
-> LEFT JOIN Volunteer_Assignments va ON v.Volunteer_ID = va.Volunteer_ID
-> WHERE va.Disaster_ID IS NULL;
```

Volunteer_ID	Name
12	Rohini Kulrkani

1 row in set (0.01 sec)

17. Disasters with no victims.

```
mysql> SELECT d.Disaster_ID, d.Disaster_Type
-> FROM Disaster d
-> LEFT JOIN Victim v ON d.Disaster_ID = v.Disaster_ID
-> WHERE v.Victim_ID IS NULL;
```

Disaster_ID	Disaster_Type
10	Forest Fire

1 row in set (0.00 sec)

18. Volunteer roles used in multiple disasters.

```
mysql> SELECT Role, COUNT(DISTINCT Disaster_ID) AS Disaster_Count
-> FROM Volunteer_Assignments
-> GROUP BY Role
-> HAVING COUNT(DISTINCT Disaster_ID) > 1;
+-----+-----+
| Role | Disaster_Count |
+-----+-----+
| Rescue | 2 |
+-----+-----+
1 row in set (0.01 sec)
```

19. Shelters that store more than 500 total items.

```
mysql> SELECT s.Shelter_ID, s.Address, SUM(si.Quantity) AS Total_Items
-> FROM Shelter s
-> JOIN Shelter_Inventory si ON s.Shelter_ID = si.Shelter_ID
-> GROUP BY s.Shelter_ID, s.Address
-> HAVING SUM(si.Quantity) > 500;
+-----+-----+-----+
| Shelter_ID | Address | Total_Items |
+-----+-----+-----+
| 1 | 123 Relief Road, Pune | 800 |
+-----+-----+-----+
1 row in set (0.00 sec)
```

20. Each disaster with total victims and total volunteers.

```
mysql> SELECT d.Disaster_Type,
-> COUNT(DISTINCT v.Victim_ID) AS Total_Victims,
-> COUNT(DISTINCT va.Volunteer_ID) AS Total_Volunteers
-> FROM Disaster d
-> LEFT JOIN Victim v ON d.Disaster_ID = v.Disaster_ID
-> LEFT JOIN Volunteer_Assignments va ON d.Disaster_ID = va.Disaster_ID
-> GROUP BY d.Disaster_Type;
+-----+-----+-----+
| Disaster_Type | Total_Victims | Total_Volunteers |
+-----+-----+-----+
| Building Collapse | 1 | 1 |
| Chemical Leak | 1 | 1 |
| Cyclone | 1 | 1 |
| Drought | 1 | 1 |
| Earthquake | 1 | 2 |
| Fire | 1 | 1 |
| Flood | 1 | 1 |
| Forest Fire | 0 | 1 |
| Landslide | 1 | 0 |
| Tsunami | 1 | 1 |
+-----+-----+-----+
10 rows in set (0.00 sec)
```

21. Procedure: Count victims in a disaster.

```
mysql> DELIMITER //
mysql> CREATE PROCEDURE CountVictims(IN p_disaster_id INT)
-> BEGIN
-> DECLARE v_count INT;
-> SELECT COUNT(*) INTO v_count FROM Victim WHERE Disaster_ID = p_disaster_id;
-> SELECT CONCAT('Total victims for disaster ', p_disaster_id, ' = ', v_count) AS Result;
-> END //
Query OK, 0 rows affected (0.16 sec)

mysql> DELIMITER ;
mysql> call CountVictims(1);
+-----+
| Result |
+-----+
| Total victims for disaster 1 = 1 |
+-----+
1 row in set (0.01 sec)

Query OK, 0 rows affected (0.01 sec)
```

22. Procedure: Volunteers assigned to a disaster.

```
mysql> DELIMITER //
mysql> CREATE PROCEDURE ShowVolunteers(IN p_disaster_id INT)
-> BEGIN
->   SELECT v.Volunteer_ID, v.Name, va.Role
->   FROM Volunteer v
->   JOIN Volunteer_Assignments va ON v.Volunteer_ID = va.Volunteer_ID
->   WHERE va.Disaster_ID = p_disaster_id;
-> END //
Query OK, 0 rows affected (0.01 sec)

mysql> DELIMITER ;
mysql> call ShowVolunteers(2);
+-----+-----+-----+
| Volunteer_ID | Name       | Role      |
+-----+-----+-----+
|          2  | Rajesh Patel | Transport |
+-----+-----+-----+
1 row in set (0.00 sec)

Query OK, 0 rows affected (0.01 sec)
```

23. Procedure: Insert a new resource.

```
mysql> DELIMITER //
mysql> CREATE PROCEDURE AddResource(IN p_id INT, IN p_name VARCHAR(100), IN p_desc VARCHAR(200))
-> BEGIN
->   INSERT INTO Resource(Resource_ID, Resource_Name, Description)
->   VALUES(p_id, p_name, p_desc);
-> END //
Query OK, 0 rows affected (0.01 sec)

mysql> DELIMITER ;
mysql> call AddResource(11, 'Clothes', 'Basic clothing kits');
Query OK, 1 row affected (0.01 sec)

mysql> select * from Resource;
+-----+-----+-----+
| Resource_ID | Resource_Name | Description |
+-----+-----+-----+
|          1  | Food Packets  | Ready-to-eat meal kits |
|          2  | Water Bottles | 1L packaged water |
|          3  | Blankets      | Warm blankets for victims |
|          4  | First Aid Kits | Basic medical supplies |
|          5  | Power Generators | Portable electric generators |
|          6  | Tents         | Temporary shelter tents |
|          7  | Clothes       | Basic clothing kits |
|          8  | Sanitation Kits | Soap, towels, sanitary pads |
|          9  | Flashlights   | Battery-powered torches |
|         10  | Fuel Cans     | Portable fuel for transport and cooking |
|         11  | Clothes       | Basic clothing kits |
+-----+-----+-----+
11 rows in set (0.00 sec)
```

24. Function: Get shelter capacity.

```
mysql> DELIMITER //
mysql> CREATE FUNCTION GetShelterCapacity(p_shelter_id INT)
-> RETURNS INT
-> DETERMINISTIC
-> BEGIN
->   DECLARE v_capacity INT;
->   SELECT Capacity INTO v_capacity FROM Shelter WHERE Shelter_ID = p_shelter_id;
->   RETURN IFNULL(v_capacity, 0);
-> END //
Query OK, 0 rows affected (0.01 sec)
```

```
mysql> SELECT GetShelterCapacity(2);
+-----+
| GetShelterCapacity(2) |
+-----+
|                300   |
+-----+
1 row in set (0.01 sec)
```

25. Procedure: Update severity based on victim count.

```
mysql> DELIMITER //
mysql> CREATE PROCEDURE UpdateSeverity(IN p_disaster_id INT)
-> BEGIN
-> DECLARE v_count INT;
-> SELECT COUNT(*) INTO v_count FROM Victim WHERE Disaster_ID = p_disaster_id;
->
-> IF v_count > 500 THEN
-> UPDATE Disaster SET Severity_Level = 'Extreme' WHERE Disaster_ID = p_disaster_id;
-> ELSEIF v_count BETWEEN 100 AND 500 THEN
-> UPDATE Disaster SET Severity_Level = 'High' WHERE Disaster_ID = p_disaster_id;
-> ELSE
-> UPDATE Disaster SET Severity_Level = 'Moderate' WHERE Disaster_ID = p_disaster_id;
-> END IF;
-> END //
Query OK, 0 rows affected (0.01 sec)

mysql> DELIMITER ;
mysql> call UpdateSeverity(1);
Query OK, 1 row affected (0.01 sec)
```

26. Procedure: List all resources per shelter.

```
mysql> DELIMITER //
mysql> CREATE PROCEDURE ListResourcesPerShelter()
-> BEGIN
-> SELECT s.Address, r.Resource_Name, si.Quantity
-> FROM Shelter_Inventory si
-> JOIN Shelter s ON si.Shelter_ID = s.Shelter_ID
-> JOIN Resource r ON si.Resource_ID = r.Resource_ID
-> ORDER BY s.Address;
-> END //
Query OK, 0 rows affected (0.01 sec)

mysql> DELIMITER ;
mysql> call ListResourcesPerShelter();
+-----+-----+-----+
| Address                                | Resource_Name | Quantity |
+-----+-----+-----+
| 123 Relief Road, Pune                 | Food Packets  | 500      |
| 123 Relief Road, Pune                 | Water Bottles | 300      |
| City Hall, Delhi                      | Tents         | 100      |
| Community Center, Vadodara            | Flashlights   | 75       |
| Green Valley Hall, Chennai            | First Aid Kits | 150      |
| Kolkata Central Camp                  | Fuel Cans     | 50       |
| Ocean View Shelter, Vizag             | Sanitation Kits | 120     |
| Peace Center, Darjeeling              | Power Generators | 20      |
| Sector 9 Camp, Mumbai                 | Blankets      | 200      |
| Sunrise School, Nagpur                | Clothes       | 80       |
+-----+-----+-----+
10 rows in set (0.00 sec)

Query OK, 0 rows affected (0.02 sec)
```

27. Procedure: Delete a volunteer.

```
mysql> DELIMITER //
mysql> CREATE PROCEDURE DeleteVolunteer(IN p_vol_id INT)
  -> BEGIN
  ->   DELETE FROM Volunteer WHERE Volunteer_ID = p_vol_id;
  -> END //
Query OK, 0 rows affected (0.01 sec)
```

```
mysql> DELIMITER ;
mysql> call DeleteVolunteer(1);
Query OK, 1 row affected (0.01 sec)
```

```
mysql> select * from Volunteer
  -> ;
```

Volunteer_ID	Name	Availability
2	Rajesh Patel	Available
3	Neha Joshi	Unavailable
4	Vikram Deshmukh	Available
5	Sara Khan	Available
6	Manoj Sharma	Available
7	Tina Mehta	Available
8	Ravi Bhosale	Unavailable
9	Nikita Jain	Available
10	Suresh Reddy	Available
12	Rohini Kulrkani	Available

```
10 rows in set (0.00 sec)
```

28. Function: Count resources assigned to a disaster.

```
mysql> DELIMITER //
mysql> CREATE FUNCTION CountResourcesByDisaster(p_disaster_id INT)
  -> RETURNS INT
  -> DETERMINISTIC
  -> BEGIN
  ->   DECLARE v_total INT;
  ->   SELECT COUNT(DISTINCT Resource_ID) INTO v_total
  ->   FROM Resource_Location WHERE Disaster_ID = p_disaster_id;
  ->   RETURN IFNULL(v_total, 0);
  -> END //
Query OK, 0 rows affected (0.01 sec)
```

```
mysql> DELIMITER ;
mysql> call CountResourcesByDisaster(2);
ERROR 1305 (42000): PROCEDURE disaster_management_system.CountResourcesByDisaster does not exist
mysql> select CountResourcesByDisaster(2);
```

CountResourcesByDisaster(2)
1

```
1 row in set (0.00 sec)
```

29. Automatically update Shelter capacity when a new Victim is assigned.

```

mysql>
mysql> CREATE TRIGGER reduce_shelter_capacity
-> BEFORE INSERT ON Victim
-> FOR EACH ROW
-> BEGIN
->     DECLARE current_capacity INT;
->     SELECT Capacity INTO current_capacity
->     FROM Shelter
->     WHERE Shelter_ID = NEW.Shelter_ID;
->     IF current_capacity IS NULL THEN
->         SIGNAL SQLSTATE '45000'
->         SET MESSAGE_TEXT = 'Shelter does not exist.';
->     END IF;
->     IF current_capacity <= 0 THEN
->         SIGNAL SQLSTATE '45000'
->         SET MESSAGE_TEXT = 'Shelter is already full. Cannot assign new victim.';
->     END IF;
->     UPDATE Shelter
->     SET Capacity = Capacity - 1
->     WHERE Shelter_ID = NEW.Shelter_ID;
-> END$$
Query OK, 0 rows affected (0.21 sec)

mysql>
mysql> DELIMITER ;
mysql> SELECT Shelter_ID, Capacity FROM Shelter WHERE Shelter_ID = 1;
+-----+-----+
| Shelter_ID | Capacity |
+-----+-----+
|          1 |        200 |
+-----+-----+
1 row in set (0.03 sec)

mysql> INSERT INTO Victim (Victim_ID, Name, Age, Contact, Disaster_ID, Shelter_ID)
-> VALUES (11, 'Rahul Tiwari', 30, '9876500000', 1, 1);
Query OK, 1 row affected (0.05 sec)

mysql> SELECT Shelter_ID, Capacity FROM Shelter WHERE Shelter_ID = 1;
+-----+-----+
| Shelter_ID | Capacity |
+-----+-----+
|          1 |        199 |
+-----+-----+
1 row in set (0.00 sec)

```

30. Log deleted disaster incidents into an archive table.




```

mysql> CREATE TABLE Incident_Archive (
->   Archive_ID INT AUTO_INCREMENT PRIMARY KEY,
->   Report_ID INT,
->   Description VARCHAR(255),
->   Timestamp DATETIME,
->   Volunteer_ID INT,
->   Disaster_ID INT,
->   Deleted_On DATETIME
-> );
Query OK, 0 rows affected (0.08 sec)

mysql>
mysql> DELIMITER $$
mysql>
mysql> CREATE TRIGGER archive_deleted_incident
-> BEFORE DELETE ON Incident_Report
-> FOR EACH ROW
-> BEGIN
->   INSERT INTO Incident_Archive (Report_ID, Description, Timestamp, Volunteer_ID, Disaster_ID, Deleted_On)
->   VALUES (OLD.Report_ID, OLD.Description, OLD.Timestamp, OLD.Volunteer_ID, OLD.Disaster_ID, NOW());
-> END$$
Query OK, 0 rows affected (0.02 sec)

mysql>
mysql> DELIMITER ;
mysql>
mysql> SELECT * FROM Incident_Report WHERE Report_ID = 5;
+-----+-----+-----+-----+-----+
| Report_ID | Description          | Timestamp          | Volunteer_ID | Disaster_ID |
+-----+-----+-----+-----+-----+
| 5         | Fire under control   | 2025-06-15 16:45:00 | 5           | 5           |
+-----+-----+-----+-----+-----+
1 row in set (0.01 sec)

mysql> DELETE FROM Incident_Report WHERE Report_ID = 5;
Query OK, 1 row affected (0.01 sec)

mysql> SELECT * FROM Incident_Archive WHERE Report_ID = 5;
+-----+-----+-----+-----+-----+-----+-----+
| Archive_ID | Report_ID | Description          | Timestamp          | Volunteer_ID | Disaster_ID | Deleted_On          |
+-----+-----+-----+-----+-----+-----+-----+
| 1          | 5         | Fire under control   | 2025-06-15 16:45:00 | 5           | 5           | 2025-11-06 12:28:02 |
+-----+-----+-----+-----+-----+-----+-----+
1 row in set (0.01 sec)

```